

Taxonomía de la Automatización Conversacional

La imagen muestra un mapa completo de **todos los tipos de sistemas conversacionales** que existen, organizados por cómo reciben información, razonan, responden, actúan y recuerdan.

Visión general: dos grandes familias

CONVERSATIONAL AUTOMATION

|

— RETRIEVAL BASED (basado en recuperación)

Respuestas predefinidas guardadas en DB, XML o TXT

→ El sistema BUSCA la respuesta correcta

|

— GENERATIVE BASED (basado en generación)

Genera texto token a token prediciendo el siguiente

→ El sistema CREA la respuesta en tiempo real

Los 6 tipos de sistemas (columnas)

Familia 1: Retrieval Based

Tipo 1 — Menú / Opciones (IVR, Quick Replies)

El sistema más simple. El usuario no escribe libremente: elige entre opciones predefinidas.

- **Input:** Opción, keyword, botón
- **Inferencia:** IF THEN ELSE — si el usuario eligió A, haz X
- **Respuesta:** Texto fijo traído de una base de datos
- **Acción:** Mapea la opción elegida a una acción
- **Memoria:** Solo datos estructurados, largo plazo

Ejemplo real: menú telefónico "Marque 1 para soporte, 2 para ventas..."

Tipo 2 — Basado en Reglas (Prolog, Rules Engine)

El usuario ya puede escribir, pero el sistema busca patrones exactos con reglas escritas a mano.

- **Input:** Una sola frase (utterance)
- **Inferencia:** Verifica reglas axiomáticas — ¿la frase cumple la regla R?
- **Respuesta:** Conjunto de respuestas predefinidas en base de datos
- **Acción:** Mapea conjuntos de reglas a acciones específicas
- **Memoria:** Datos estructurados, largo plazo

Ejemplo real: chatbot bancario antiguo con árbol de decisión escrito manualmente

Tipo 3 — Basado en Intención / Intent (Dialogflow, LUIS) ← *señalado con flecha roja*

El más avanzado de los retrieval. Usa ML/DL para entender la **intención** del usuario aunque lo diga de distintas maneras.

- **Input:** Contexto + frase del usuario
- **Inferencia:** Clasifica la intención aprendida (no programada a mano)
- **Respuesta:** Respuestas predefinidas, pero seleccionadas por intención detectada
- **Acción:** Mapea la intención aprendida a la acción correspondiente
- **Memoria:** Mayormente estructurada, corto y largo plazo

Ejemplo real: chatbot de atención al cliente con Dialogflow que entiende "quiero cancelar mi suscripción" y "baja mi cuenta" como la misma intención

Familia 2: Generative Based

A partir de aquí el sistema **genera** la respuesta, no la busca. Usa LLMs (GPT, Claude, Gemini...).

Tipo 4 — Generativo puro (solo Input Prompt) ⚠ **RIESGO**

El usuario escribe directamente al LLM sin ningún sistema de control.

- **Input:** Solo el prompt del usuario
- **Inferencia:** Predicción de tokens aprendida durante entrenamiento
- **Respuesta:** Texto generado, dominio abierto (puede decir cualquier cosa)

- **Acción:** ⚠️ **"Risky, not Safe"** — sin restricciones, el LLM puede decidir acciones impredecibles
- **Memoria:** Datos no estructurados, solo corto plazo

Ejemplo: darle acceso a herramientas a un LLM sin ningún system prompt ni instrucciones de seguridad → puede tomar decisiones peligrosas

Tipo 5 — Generativo con Contexto (Input Prompt + Context)

Se agrega contexto (historial, documentos, datos del usuario) junto al prompt.

- **Input:** Prompt del usuario + contexto relevante
- **Inferencia:** Predicción de tokens con más información de fondo
- **Respuesta:** Texto generado, específico al contexto dado
- **Acción:** Actúa bajo su propia autonomía (con más información que el tipo 4)
- **Memoria:** Datos estructurados y no estructurados, corto y largo plazo

Ejemplo: chatbot de soporte que recibe el historial del ticket y el manual del producto junto al mensaje del usuario

Tipo 6 — Generativo completo (System Prompt + Context + Input Prompt)

← **señalado con flecha roja**

El sistema más potente y controlado. Combina instrucciones del sistema, contexto y el prompt del usuario.

- **Input:** System Prompt + Contexto + Prompt del usuario
- **Inferencia:** Predicción de tokens guiada por instrucciones del sistema
- **Respuesta:** Texto generado, dominio cerrado o específico (el system prompt lo delimita)
- **Acción:** Actúa bajo su propia autonomía, pero dentro de las reglas del system prompt
- **Memoria:** Datos estructurados y no estructurados, siempre, corto y largo plazo

Ejemplo: un agente empresarial que tiene un system prompt con su rol, restricciones, tono y herramientas disponibles

Tabla resumen comparativa

Dimensión	Menú/Opción	Reglas	Intent (ML)	Gen. Puro	Gen. + Contexto
Input	Keywords/botones	Frase suelta	Contexto + frase	Prompt	Prompt + Contexto
Inferencia	IF/ELSE	Reglas axiomáticas	Clasificación de intención	Token prediction	Token prediction
Respuesta	Fija de BD	Predefinida de BD	Predefinida de BD	Generada (abierta)	Generada (específica)
Acción	Opción → acción	Regla → acción	Intención → acción	⚠ Riesgosa	Autónoma
Memoria	Estructurada / LP	Estructurada / LP	Mayormente est. / CP+LP	No estructurada / CP	Est. + No est. / CP+LP

LP = Largo Plazo, CP = Corto Plazo

El mensaje clave del profesor: las dos flechas rojas

Las flechas señalan el **tipo 3** y el **tipo 6** porque son los más sofisticados de cada familia:

	Retrieval → Tipo 3 (Intent)	Generative → Tipo 6 (Completo)
Por qué es el mejor de su familia	Aprende intenciones, no necesita reglas manuales	Tiene control (system prompt) + potencia (LLM) + contexto
Limitación	Las respuestas siguen siendo predefinidas	Requiere diseño cuidadoso del system prompt
Cuándo usarlo	Flujos acotados con respuestas conocidas	Agentes complejos con razonamiento y herramientas

El cuadro rojo (advertencia): Generativo Puro sin control

La celda "Risky, not Safe" en el Tipo 4 es una advertencia importante:

Un LLM con acceso a herramientas pero **sin system prompt y sin contexto** puede tomar acciones impredecibles porque no tiene restricciones ni rol definido.

Esto conecta directamente con el diseño de agentes seguros:

```
LLM solo          → potente pero impredecible
LLM + system prompt → potente Y controlado
LLM + system prompt + context + tools = AGENTE bien diseñado
```

Evolución histórica de izquierda a derecha

1990s	2000s	2010s	2020s
Menú IVR →	Rules Engine →	Dialogflow/LUIS →	LLMs / Agentes
(estático)	(semi-flexible)	(ML clasificador)	(generativo)

Cada salto implica:

- Más **flexibilidad** en el input que acepta
- Más **autonomía** en las decisiones que toma
- Más **riesgo** si no se diseña bien
- Más **potencia** si se diseña correctamente

Relación con los Agentes de IA

El **Tipo 6** (System Prompt + Context + Input Prompt) es la base de los agentes modernos:

- El **System Prompt** define el rol, las herramientas disponibles y los límites
- El **Contexto** provee memoria, documentos y estado actual
- El **Input Prompt** es la tarea concreta del usuario
- La **acción autónoma** es lo que convierte al sistema en un agente real

Ver también:

[../Modulo1_Introduccion_Motivacion/Sesion01_Introduccion_Agentes/que_es_un_agente_ia.md](#)

Por qué la última columna ES un Agente de IA — análisis fila por fila

El profesor señaló explícitamente: **"la última columna de la derecha es un Agente"**. A continuación se explica por qué cada fila de esa columna —y solo la combinación de todas ellas— produce un verdadero agente.

La definición de referencia (Gartner) dice que un agente **percibe, decide, actúa y logra objetivos** de forma autónoma. Veamos cómo cada fila de la última columna cumple exactamente eso.

Fila 1 — INPUT: System Prompt + Context + Input Prompt → el agente *PERCIBE*

La última columna es la única que tiene **tres capas de entrada simultáneas**. Cada capa cumple un rol distinto e insustituible:

Capa	Qué aporta	Sin ella, qué pasa
System Prompt	Identidad, rol, herramientas disponibles, restricciones, dominio	El agente no sabe quién es ni qué puede hacer → acciones riesgosas (problema del Tipo 4)
Context	Estado actual del mundo: historial de conversación, documentos, datos del usuario, resultados de herramientas previas	El agente no sabe qué pasó antes ni en qué situación está → no puede razonar con coherencia
Input Prompt	La tarea concreta que el usuario quiere resolver ahora	El agente no tiene objetivo → no hay nada que lograr

Esta trinidad — **identidad + situación + tarea** — es lo que le permite al agente *percibir* su entorno de forma completa, igual que lo describe Gartner.

Ninguna columna anterior tiene las tres capas:

- Tipos 1-3 (retrieval): no tienen ni System Prompt ni generación, sus "inputs" son rígidos
- Tipo 4: solo Input Prompt → percepción mínima, sin identidad ni contexto
- Tipo 5: Input Prompt + Context → mejor, pero sin System Prompt no tiene identidad ni herramientas definidas

Tipo 4: [Input Prompt] → percepción incompleta
Tipo 5: [Context] + [Input Prompt] → percepción parcial
Tipo 6: [System Prompt] + [Context] + [Input Prompt] → percepción completa ✓

Fila 2 — INFERENCE: Generative LLMs + Token Prediction → *el agente RAZONA*

La inferencia del Tipo 6 tiene **dos niveles**:

Nivel 1 — el tipo de modelo: GENERATIVE SML, LLMs, GPTs

- No usa reglas escritas a mano (como los Tipos 1-2)
- No clasifica intenciones en categorías fijas (como el Tipo 3)
- Razona de forma **emergente**: puede manejar situaciones que nunca vio durante el entrenamiento

Nivel 2 — el mecanismo: Learnt by tokens – TOKEN PREDICTION

- El modelo aprendió de cantidades masivas de texto y sabe cómo razonar paso a paso
- Cada token que genera es una "decisión": el modelo elige la continuación más coherente dado todo lo que recibió en el input
- Esto le permite: planificar, descomponer tareas, encadenar razonamientos, detectar cuándo necesita usar una herramienta

La clave: en los sistemas retrieval, el razonamiento está **pre-programado por un humano**. En el Tipo 6, el razonamiento está **aprendido por el modelo** y se aplica de forma flexible a cualquier situación.

```
Tipo 1: IF opción == A → acción A      (razonamiento: 0 líneas de código)
Tipo 2: IF regla cumplida → respuesta R (razonamiento: árbol de decisión
estático)
Tipo 3: clasificar_intención(texto) → R (razonamiento: clasificador ML,
categorías fijas)
Tipo 6: razonar_desde_contexto_completo() → plan flexible ✓
```

Fila 3 — REPLY: Generated tokens + Closed/Domain Specific → *el agente COMUNICA con control*

Esta fila tiene dos subceldas que juntas resuelven una tensión fundamental:

Subcelda 1: Generated tokens

- La respuesta **no existe en ninguna base de datos**: se crea en tiempo real
- El agente puede formular respuestas para situaciones completamente nuevas
- Esto es lo que diferencia a los sistemas generativos de todos los retrieval

Subcelda 2: Text is generated from prior leaning (closed domain or Domain Specific)

- "Closed domain" = el System Prompt restringe al agente a un dominio concreto
- El agente no divaga, no inventa fuera de su rol, no responde sobre temas prohibidos
- Esta restricción es el **antídoto al Tipo 4** (que era Open Domain → riesgoso)

Comparación directa:

Tipo	Respuesta	Dominio	Problema
Tipos 1-3	Predefinida (de BD)	Fijo	No puede manejar lo inesperado
Tipo 4	Generada	Abierto (Open Domain)	Puede decir o hacer cualquier cosa
Tipo 5	Generada	Context Specific	Mejor, pero sin System Prompt pierde consistencia
Tipo 6	Generada	Cerrado/Específico	Flexible Y controlado ✓

La combinación generación + dominio cerrado es lo que hace al agente **capaz** (no busca respuestas) y **confiable** (no se sale de su rol).

Fila 4 — ACTION: Autonomy + Textual description of tools → el agente ACTÚA

Esta es la fila **más importante** para entender por qué el Tipo 6 es un agente. Tiene dos subceldas críticas:

Subcelda 1: Status of action (on its Autonomy)

"On its Autonomy" significa que **el agente decide por sí solo** cuándo actuar, qué acción tomar y cómo ejecutarla. Compara esto con todos los demás:

Tipo	¿Quién decide la acción?
Tipo 1 (Menú)	El USUARIO elige la opción → el sistema ejecuta
Tipo 2 (Reglas)	El PROGRAMADOR escribió la regla → el sistema la sigue
Tipo 3 (Intent)	El ML clasifica la intención → el PROGRAMADOR mapeó qué acción corresponde
Tipo 4 (Gen. puro)	El LLM decide → pero sin restricciones → ⚠ riesgoso
Tipo 6 (Agente)	El LLM decide → dentro de los límites del System Prompt → ✓ autónomo Y seguro

La autonomía en los tipos 1-3 es **falsa**: siempre hay un humano (usuario o programador) que predefinió cada posible acción. El Tipo 6 actúa en situaciones que nadie pre-programó.

Subcelda 2: Textual description of tools and how to take action

Esta subcelda es técnicamente revolucionaria. En los sistemas retrieval, las herramientas están **hard-codeadas** en el software:

```
# Sistema retrieval (Tipo 2/3)
if intent == "cancelar_suscripcion":
    call_api("cancel", user_id) # el programador escribió esto
```

En el Tipo 6, las herramientas se le **describen al agente en lenguaje natural** dentro del System Prompt:

```
# Sistema agente (Tipo 6) – fragmento de System Prompt
Tienes acceso a las siguientes herramientas:
- cancel_subscription(user_id): cancela la suscripción del usuario
- send_email(to, subject, body): envía un email
- query_database(sql): consulta la base de datos
```

El agente **lee, entiende y decide** cuándo usar cada herramienta. Consecuencias de esto:

1. Puedes darle herramientas nuevas **sin reprogramar** — solo agregas la descripción al System Prompt
2. El agente puede **encadenar herramientas** para resolver tareas complejas (buscar → procesar → enviar resultado)
3. El agente puede **decidir no usar** una herramienta si no es necesaria
4. El agente puede **elegir entre varias herramientas** según la situación

Esto es lo que en la industria se llama **"tool use"** o **"function calling"** — y es el corazón técnico de los agentes modernos.

```
Sistema retrieval:  herramienta ←hard-coded→ acción
Sistema agente:     herramienta ←descrita en texto→ LLM decide si/cuándo/cómo
usarla ✓
```

Fila 5 — MEMORY: Structured + Unstructured, Always, Long and Short Term → *el agente RECUERDA*

La palabra **"Always"** (siempre) es lo que distingue esta celda de todas las demás.

Tipo	Memoria estructurada	Memoria no estructurada	Plazo
Tipo 1	Sí	No	Solo largo
Tipo 2	Sí	No	Solo largo
Tipo 3	Mayormente sí	Muy poco	Corto y largo
Tipo 4	No (mayormente)	Sí	Solo corto
Tipo 5	Posiblemente	Posiblemente	Corto y largo
Tipo 6	Sí, siempre	Sí, siempre	Corto Y largo

¿Qué es cada tipo de memoria?

- **Estructurada:** bases de datos, APIs, perfiles de usuario, inventarios, registros con esquema definido
- **No estructurada:** historial de conversación, documentos PDF, emails, páginas web, texto libre

¿Por qué necesita ambas el agente?

Un agente real trabaja en el mundo real, donde la información viene en ambos formatos:

- Para saber si hay stock de un producto → base de datos estructurada
- Para entender el contexto de un reclamo de cliente → conversación (no estructurada)
- Para seguir una política de la empresa → documento PDF (no estructurado)
- Para registrar una transacción → tabla SQL (estructurada)

¿Por qué necesita ambos plazos?

- **Corto plazo:** lo que pasó en esta sesión — si el usuario ya dijo su nombre, el agente no debe volver a preguntarlo
- **Largo plazo:** lo que pasó en sesiones anteriores — si el usuario tiene historial de compras, el agente puede personalizar sus respuestas

Un sistema sin memoria larga no puede **aprender del pasado**. Sin memoria corta, no puede **mantener coherencia dentro de una tarea**. Un agente necesita ambas para funcionar como tal.

El argumento completo: por qué SOLO el Tipo 6 es un Agente

Recapitulando la definición de Gartner aplicada fila por fila:

Capacidad del agente	Fila del cuadro	Cómo la cumple el Tipo 6
Percibir	INPUT	Tres capas: identidad (System Prompt) + situación (Context) + tarea (Input Prompt)
Razonar / Decidir	INFERENCE	LLM con token prediction: razonamiento flexible y emergente, no programado
Comunicar	REPLY	Genera texto nuevo (no recupera) dentro de un dominio controlado
Actuar	ACTION	Autonomía real + herramientas descritas en lenguaje natural (tool use)
Recordar / Mantener estado	MEMORY	Datos estructurados y no estructurados, corto y largo plazo, siempre

Ninguna otra columna cumple las cinco. Todos los sistemas anteriores fallan en al menos una:

Tipo 1 → no razona, no genera, no actúa autónomamente

Tipo 2 → no razona, no genera, acciones hard-codeadas

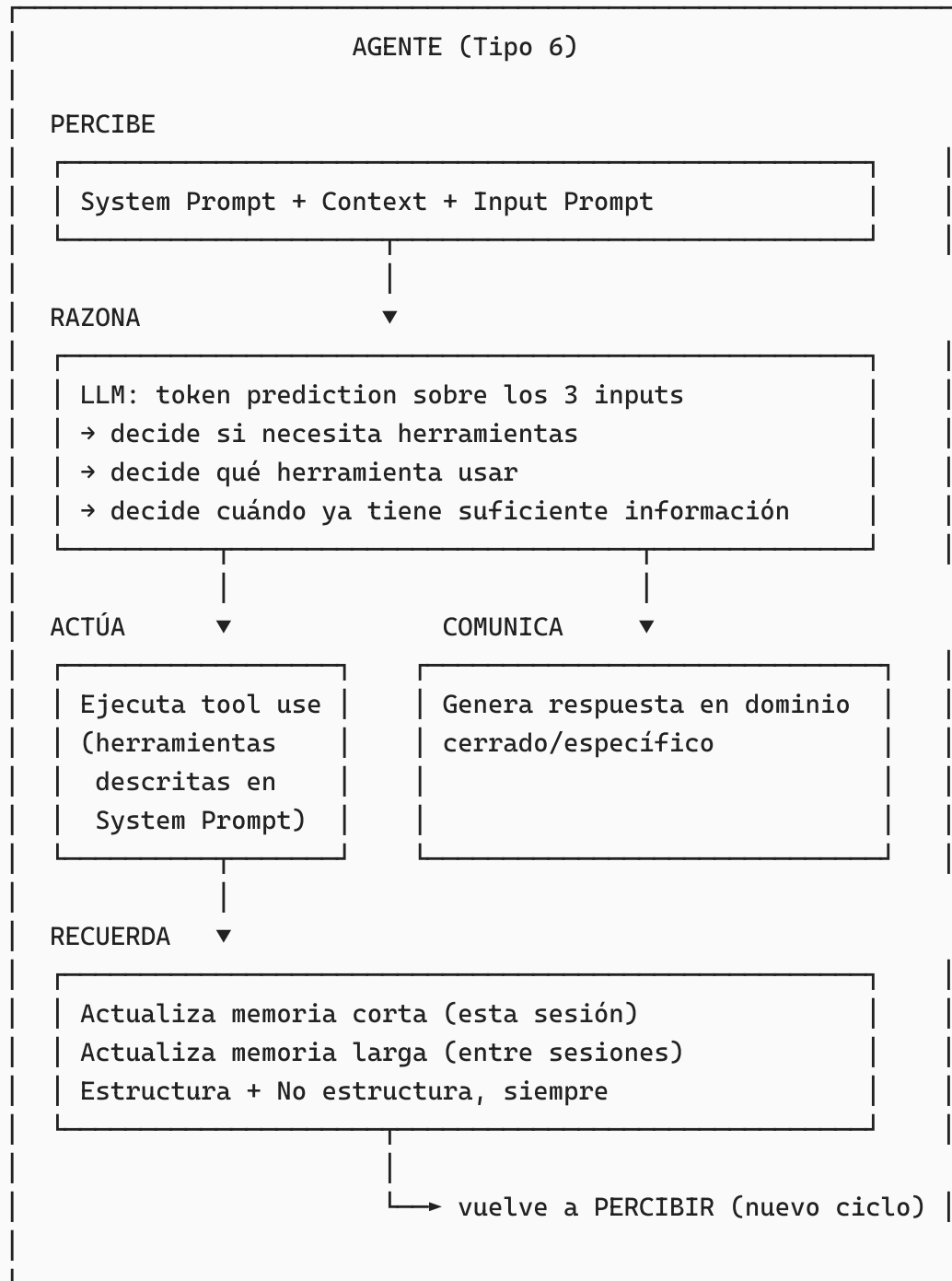
Tipo 3 → genera intención pero respuestas son predefinidas, acciones mapeadas

Tipo 4 → razona y actúa, pero sin identidad ni contexto → peligroso

Tipo 5 → razona y actúa, pero sin System Prompt → sin identidad ni tools definidos

Tipo 6 → cumple las 5 dimensiones → ES un Agente ✓

El ciclo completo del Agente (Tipo 6 en funcionamiento)



El agente no termina después de una respuesta: **itera** hasta lograr el objetivo definido en el Input Prompt, usando su memoria para mantener coherencia a lo largo del proceso.

Ver también:

```
../Modulo1_Introduccion_Motivacion/Sesion01_Introduccion_Agentes/que_es_un_age
nte_ia.md
```